Séance 03 - Correction et guide enseignant

Table of Contents

1. Intentions pédagogiques . . . 1
2. Points de vigilance . . . 2
3. Correction - Réactivation . . . 2
4. Correction - Activité 1 . . . 2
5. Correction - Activité 2 . . . 3
   5.1. Résultats de `comparaisons.php` . . . 3
   5.2. Égalité stricte . . . 3
6. Correction - Activité 3 . . . 3
   6.1. Alerte disque . . . 3
   6.2. État du serveur . . . 3
7. Correction - Activité 4 . . . 4
8. Correction - Activité 5 . . . 4
   8.1. Disponibilité . . . 4
   8.2. Alerte sur les ressources . . . 5
9. Correction - Activité 6 . . . 5
10. Correction - Activité 7 . . . 6
11. Correction - Exercice autonome . . . 7
12. Correction - Travail sur papier . . . 7
13. Correction - Synthèse . . . 8
14. Correction - Exercices bonus . . . 8
   14.1. Exercice bonus 1 `[*]` - Fenêtre de maintenance . . . 8
   14.2. Exercice bonus 2 `[**]` - Éligibilité à une sauvegarde . . . 8
   14.3. Exercice bonus 3 `[*]` - Défi : prévoir la sortie . . . 9
15. Validation Git attendue . . . 9
16. Allègement par rapport à l'ancien support . . . 10

1. Intentions pédagogiques

Cette séance extrait les conditions de l'ancien TP qui les associait aux formulaires, à `$_POST`, aux tableaux et à `foreach`. Ces notions sont désormais séparées afin que les étudiants consacrent une séance entière au raisonnement booléen et aux structures de décision.

La progression interne est volontairement graduelle :

1. produire une valeur booléenne ;

2. exécuter une instruction avec `if` ;
3. choisir entre deux branches ;
4. classer une valeur avec `elseif` ;
5. combiner des conditions ;
6. appliquer plusieurs règles ordonnées ;
7. corriger des erreurs logiques.

Sont exclus de cette séance : formulaires, superglobales, tableaux, boucles, fonctions, opérateur ternaire et `match`.

2. Points de vigilance

- Ne pas transformer la séance en cours sur Docker : l'environnement a été installé et expliqué à la séance 02.
- Faire prévoir les résultats avant chaque exécution.
- Insister sur les valeurs aux frontières : `70`, `90`, `2` et `4`.
- Faire distinguer erreur de syntaxe et erreur logique.
- Ne pas afficher le contenu complet de la zone d'index : `git status` suffit après l'indexation.
- Vérifier que les étudiants n'écrivent pas plusieurs `if` indépendants lorsqu'une seule classification est attendue.
- Privilégier `===` et `!==` dans les exemples.

3. Correction - Réactivation

1. `bool`
2. `.`
3. `210`
4. `var_dump()`
5. L'affectation stocke la valeur `true` dans la variable ; `echo` tente d'afficher sa valeur et produit généralement `1`.

4. Correction - Activité 1

Commandes attendues :

```
git status
git pull
```

Puis, depuis le dossier du TP :

```
docker compose up -d
```

Les fichiers `.php` (dont `diagnostic.php`, déjà fourni) vont dans le dossier du TP. Contrôle : `http://localhost:8181/`.

5. Correction - Activité 2

5.1. Résultats de `comparaisons.php`

```
bool(true)
bool(false)
bool(true)
bool(true)
```

5.2. Égalité stricte

```
$codeEtat = "1";

var_dump($codeEtat == 1);   // bool(true)
var_dump($codeEtat === 1);  // bool(false)
```

`==` compare après conversion éventuelle des types. `===` impose la même valeur et le même type. Le module privilégie `===` et `!==`.

6. Correction - Activité 3

6.1. Alerte disque

La condition correcte est :

```
if ($espaceLibreGo <= $seuilAlerteGo) {
    echo "ALERTE : espace disque insuffisant sur " . $nomServeur . ".";
}
```

La valeur `10` déclenche l'alerte parce que l'opérateur `⇐` inclut l'égalité.

6.2. État du serveur

```
if ($estActif) {
    echo $nomServeur . " est actif.";
```

```
} else {
    echo $nomServeur . " est arrêté.";
}
```

`$estActif` est déjà un booléen. L'expression peut donc être utilisée directement comme condition.

7. Correction - Activité 4

```
<?php

$chargeCpu = 75;

if ($chargeCpu >= 90) {
    $etat = "CRITIQUE";
} elseif ($chargeCpu >= 70) {
    $etat = "SURVEILLANCE";
} else {
    $etat = "NORMAL";
}

echo "Charge CPU : " . $chargeCpu . " %<br>";
echo "État : " . $etat;
```

Charge CPU	État
95	CRITIQUE
90	CRITIQUE
75	SURVEILLANCE
70	SURVEILLANCE
69	NORMAL
20	NORMAL

Il faut tester `>= 90` avant `>= 70`, car toute valeur supérieure ou égale à 90 vérifie également la condition `>= 70`. La première branche vraie arrête la chaîne `if/elseif/else`.

8. Correction - Activité 5

8.1. Disponibilité

Le service est disponible uniquement lorsque les deux booléens valent `true`.

```
if ($serveurActif && $serviceDemarre) {
    echo "Le service est disponible.";
} else {
    echo "Le service est indisponible.";
```

```
}
```

8.2. Alerte sur les ressources

```php
<?php

$chargeCpu = 50;
$memoireLibreGo = 1.5;

if ($chargeCpu >= 90 || $memoireLibreGo < 2) {
    echo "ALERTE CRITIQUE";
} else {
    echo "Aucune alerte critique";
}
```

`||` est nécessaire, car une seule ressource au-delà de son seuil suffit à déclencher l'alerte. Avec `&&`, les deux anomalies devraient être présentes simultanément.

9. Correction - Activité 6

```php
<?php

$nomServeur = "srv-bdd-01";
$estActif = true;
$modeMaintenance = false;
$chargeCpu = 75;
$memoireLibreGo = 8;

if (!$estActif) {
    $etatGlobal = "HORS SERVICE";
} elseif ($modeMaintenance) {
    $etatGlobal = "MAINTENANCE";
} elseif ($chargeCpu >= 90 || $memoireLibreGo < 2) {
    $etatGlobal = "CRITIQUE";
} elseif ($chargeCpu >= 70 || $memoireLibreGo < 4) {
    $etatGlobal = "SURVEILLANCE";
} else {
    $etatGlobal = "NORMAL";
}

echo "Serveur : " . $nomServeur . "<br>";
echo "État global : " . $etatGlobal;
```

La maintenance est testée avant les seuils, car elle décrit un état volontaire qui doit remplacer les alertes de ressources pendant l'intervention. De même, un serveur arrêté est classé `HORS SERVICE` avant toute analyse de ses mesures.

10. Correction - Activité 7

Le fichier de départ est syntaxiquement valide mais contient quatre erreurs logiques.

Erreur	Correction	Explication
`$etatServeur = "actif"`	`$etatServeur === "actif"`	`=` affecte une valeur ; `===` compare strictement.
`$chargeCpu > 90`	`$chargeCpu >= 90`	La valeur frontière `90` doit être critique.
`$chargeCpu >= 90 && $memoireLibreGo < 2`	`$chargeCpu >= 90 || $memoireLibreGo < 2`	Une seule ressource critique suffit.
`if (!$modeMaintenance)`	`if ($modeMaintenance)`	Le message doit apparaître seulement lorsque le mode vaut `true`.

Correction complète :

```
<?php

$etatServeur = "actif";
$chargeCpu = 90;
$memoireLibreGo = 1.5;
$modeMaintenance = false;

if ($etatServeur === "actif") {
    echo "Serveur actif.<br>";
} else {
    echo "Serveur inactif.<br>";
}

if ($chargeCpu >= 90) {
    echo "Charge CPU critique.<br>";
} else {
    echo "Charge CPU non critique.<br>";
}

if ($chargeCpu >= 90 || $memoireLibreGo < 2) {
    echo "Ressources critiques.<br>";
} else {
    echo "Ressources non critiques.<br>";
}

if ($modeMaintenance) {
    echo "Maintenance en cours.<br>";
} else {
    echo "Aucune maintenance.<br>";
}
```

11. Correction - Exercice autonome

```php
<?php

$utilisateurAdministrateur = true;
$serveurActif = true;
$modeMaintenance = true;
$espaceLibreGo = 15;

if (!$utilisateurAdministrateur) {
    $decision = "ACCÈS REFUSÉ";
} elseif (!$serveurActif) {
    $decision = "SERVEUR ARRÊTÉ";
} elseif (!$modeMaintenance) {
    $decision = "MODE MAINTENANCE REQUIS";
} elseif ($espaceLibreGo < 10) {
    $decision = "ESPACE DISQUE INSUFFISANT";
} else {
    $decision = "MISE À JOUR AUTORISÉE";
}

echo $decision;
```

Jeux de test minimaux :

Admin	Actif	Maintenance	Espace	Résultat
`false`	`true`	`true`	`15`	`ACCÈS REFUSÉ`
`true`	`false`	`true`	`15`	`SERVEUR ARRÊTÉ`
`true`	`true`	`false`	`15`	`MODE MAINTENANCE REQUIS`
`true`	`true`	`true`	`5`	`ESPACE DISQUE INSUFFISANT`
`true`	`true`	`true`	`15`	`MISE À JOUR AUTORISÉE`

12. Correction - Travail sur papier

Avec les valeurs initiales :

1. `!$serveurActif` donne `false` ;
2. `$chargeCpu >= 90` donne `true` ; la partie droite du `||` n'a pas besoin de modifier le résultat global ;
3. la branche critique est exécutée ;
4. le texte affiché est `CRITIQUE`.

Avec `$chargeCpu = 65` et `$memoireLibreGo = 3` :

- le test critique est faux ;
- le test de surveillance est vrai grâce à `$memoireLibreGo < 4` ;

- l'état est `SURVEILLANCE`.

Avec `&&` dans le test critique et les valeurs initiales, la condition critique devient fausse, car `3 < 2` est faux. Le test suivant est vrai et l'état obtenu devient `SURVEILLANCE`.

13. Correction - Synthèse

1. `true`, `false`
2. `=`
3. `===`
4. `!==`
5. `&&`
6. `||`
7. `!`
8. `if`, `else`
9. la condition la plus restrictive ou le seuil le plus élevé

14. Correction - Exercices bonus

14.1. Exercice bonus 1 [*] - Fenêtre de maintenance

```
<?php

$heure = 3;

if ($heure >= 22 || $heure <= 5) {
    echo "Maintenance autorisée (créneau de nuit)";
} else {
    echo "Maintenance interdite en journée";
}
```

Sorties : `3` et `23` → autorisé ; `6` et `14` → interdit. Le créneau « passe minuit » : il réunit deux plages (22–23 et 0–5) dont une seule doit être vraie, d'où le `||`. Avec `&&`, aucune heure ne pourrait être à la fois `>= 22` et `<= 5` : la maintenance serait toujours interdite.

14.2. Exercice bonus 2 [**] - Éligibilité à une sauvegarde

```
<?php

$serveurActif = true;
```

```php
$modeMaintenance = false;
$espaceLibreGo = 25;

if ($serveurActif && !$modeMaintenance && $espaceLibreGo >= 20) {
    echo "Sauvegarde lancée";
} else {
    echo "Sauvegarde reportée";
}
```

Seule la première ligne du tableau (`true`, `false`, `25`) déclenche la sauvegarde ; dès qu'une des trois conditions est fausse, le `&&` rend l'ensemble faux. Le `!` traduit « pas en maintenance » : `!$modeMaintenance` vaut `true` quand `$modeMaintenance` vaut `false`.

14.3. Exercice bonus 3 [*] - Défi : prévoir la sortie

```
bool(true)
bool(true)
bool(false)
bool(true)
bool(true)
```

Explications :

1. `$a > 0 && $b > 0` : `true && true` → `true`.
2. `$a > 0 || $b > 100` : `true || false` → `true` (le `||` s'arrête au premier vrai).
3. `!($a === 5)` : `$a === 5` est `true`, donc `!true` → `false`.
4. `$a !== "5"` : l'entier `5` et la chaîne `"5"` diffèrent par le type → `true`.
5. `$a > 0 && $b < 0 || $b === 10` : le `&&` est prioritaire sur le `||`. On évalue d'abord `($a > 0 && $b < 0)` = `true && false` = `false`, puis `false || ($b === 10)` = `false || true` = `true`.

Guide enseignant : la dernière ligne illustre la **précédence** de `&&` sur `||` (comme `*` avant `+` en arithmétique). En cas de doute, encourager les parenthèses explicites. La ligne 4 rappelle l'intérêt de `!==` (comparaison stricte de type) vu à l'activité 2.

15. Validation Git attendue

Deux commits nouveaux doivent apparaître :

```
Ajoute la classification de l'état des serveurs
Ajoute les premières conditions PHP
```

Après `git push`, `git status` doit indiquer un espace de travail propre.

16. Allègement par rapport à l'ancien support

Le support d'origine traitait dans une même séance les formulaires `POST`, les tableaux issus des cases à cocher, `isset()`, `foreach` et les conditions. Les conditions apparaissaient principalement pour choisir un lien ou gérer l'absence de langue.

La nouvelle séance conserve :

- `if`, `else` et `elseif` ;
- la distinction entre `=` et un opérateur d'égalité ;
- l'utilisation de conditions dans un contexte métier.

Elle ajoute :

- les expressions booléennes observées avec `var_dump()` ;
- les opérateurs stricts `===` et `!==` ;
- les valeurs frontières ;
- les opérateurs `&&`, `||` et `!` ;
- l'ordre des règles ;
- des jeux de tests ;
- un diagnostic d'erreurs logiques ;
- un exercice manuscrit ;
- l'utilisation systématique de Git et de Docker.